

Demystifying Data Structures: Building from Scratch

CMPT 211: Introduction to Software Development

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

The Illusion of Python's "Magic"

Python makes our lives easy. When you type `my_list.append(5)`, the list magically grows. When you type `my_list.insert(0, 5)`, everything magically shifts.

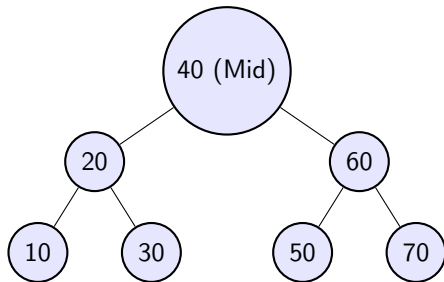
But there is no magic in Computer Science.

- Underneath Python's helpful methods is raw, rigid computer memory (RAM).
- True software engineering is understanding how to manipulate that rigid memory to build complex, dynamic behaviors yourself.
- Today, we are going to strip away Python's magic. You are going to see that **you have the power to build any complex data structure in the world** using nothing but raw arrays and integer pointers!

The Connection: Binary Search and Binary Trees

We recently wrote a Binary Search algorithm. It divides a sorted list in half, checks the midpoint, and recursively searches the left or right half.

Look closely at what happens when we run Binary Search on the array: [10, 20, 30, 40, 50, 60, 70]



A Binary Search on an array is literally just navigating a perfectly balanced Binary Search Tree (BST) hidden in the array indices!

Simulating Raw Memory (RAM)

Languages like C and C++ use **Pointers**—variables that hold actual hexadecimal physical memory addresses (e.g., 0x7FFF).

Python hides this from us. So, to build structures from scratch, we will simulate a block of RAM using a single Python list.

The Rules of our Simulation:

- 1 We create a fixed-size list: `memory = [None] * 10`
- 2 We are **forbidden** from using `.append()`, `.insert()`, or `.pop()`.
- 3 We use the **List Indices (0 to 9)** as our Pointers!

Building a Stack (LIFO) from Scratch

A Stack requires us to push and pop from the "top". We achieve this by moving a `top_pointer` up and down our memory array.

```
class RawStack:
    def __init__(self, capacity):
        self.memory = [None] * capacity    # Pre-allocating rigid memory
        self.top_pointer = -1              # -1 means the stack is empty

    def push(self, value):
        self.top_pointer += 1              # Move pointer UP
        self.memory[self.top_pointer] = value # Write to memory

    def pop(self):
        value = self.memory[self.top_pointer] # Read from memory
        self.memory[self.top_pointer] = None # Erase it
        self.top_pointer -= 1              # Move pointer DOWN
        return value
```

Building a Queue (FIFO) from Scratch

A Queue requires two pointers: one to track where the line starts (head) and one to track where new people join (tail).

```
class RawQueue:
    def __init__(self, capacity):
        self.memory = [None] * capacity
        self.head = 0    # Where we Dequeue (Read)
        self.tail = 0    # Where we Enqueue (Write)

    def enqueue(self, value):
        self.memory[self.tail] = value    # Write at the tail
        self.tail += 1                    # Move tail right

    def dequeue(self):
        value = self.memory[self.head]    # Read from the head
        self.memory[self.head] = None    # Erase it
        self.head += 1                    # Move head right
        return value
```

Notice: The data physically crawls to the right across the RAM array!

The Ultimate Test: A Linked List

A Linked List does not require data to be stored contiguously side-by-side. The data can be scattered anywhere in RAM, as long as each node contains a "Pointer" to the next node's index!

Node Format: [Value, Next_Index] (*We use -1 for Null/End*)

```
# Memory Pool: Data is completely scattered!
memory = [
    [20, -1], # Index 0: Value is 20, points nowhere (-1)
    [5, 0],  # Index 1: Value is 5, points to Index 0
    [10, 1]  # Index 2: Value is 10, points to Index 1
]

# The Head Pointer tells us where to start reading!
head_pointer = 2
```

If we start at `head_pointer = 2`, we read **10**. It points to Index 1. We jump to Index 1 and read **5**. It points to Index 0. We jump to Index 0 and read **20**. It points to -1 (Stop).

Logical Output Sequence: 10 → 5 → 20

Worst-Case Search Time: Big O Complexity

We can build these structures, but how efficient are they when we need to **search** for a specific target value?

Data Structure	Worst-Case Search	Why?
Linear Array	$O(n)$	Must check every single element.
Linked List	$O(n)$	Must hop from node to node sequentially.
Stack / Queue	$O(n)$	Must pop/dequeue everything to find it.
Sorted Array (Binary Search)	$O(\log n)$	Divides search space in half each step.
Balanced BST	$O(\log n)$	Divides search space in half each step.
Unbalanced BST	$O(n)$	Degenerates into a Linked List!

Conclusion: The way we structure our pointers drastically changes the mathematical limits of our software's speed!

Any Questions?

You now have the conceptual tools to build any data structure in existence.